TypeScript_7_8

インタフェース

インターフェースは、関連名前でオブジェクトの型を定義する一つの方法である。

```
interface Me {
    age: number;
    name: string;
}
```

属性

```
interface Book {
  author?: string;
  pages: number;
}

const ok: Book = {
  author: "Rita Dove",
  pages: 80,
};

const missing: Book = {
  pages: 80,
};
```

このように必須属性 である pages と、選択的な属性である author を作成することができる。
選択的な属性 には？を付ける。

```typescript
interface Page {
    readonly text: string;
}

function read(page: Page) {
    console.log(page.text);

    page.text += "!";

}
```

readonly は型システム上にのみ存在し、インターフェースでのみ使用できる。

```typescript
interface HasBothFunctionTypes {
  property: () => string;
  method(): string;
}

const hasBoth: HasBothFunctionTypes = {
  property: () => "",
  method() {
    return "";
  },
};

hasBoth.property();
hasBoth.method();
```

TypeScript はインターフェースを関数として宣言する方法が二つある。

メソッド：member(): void

属性： member: () ⇒

インタフェースの拡張

v6.0.3 Run Export Share

```typescript
interface Writing {
    title: string;
}

interface Novel extends Writing {
    pages: number;
}

let myNovel: Novel = {
    pages: 195,
    title: "Ant",
};

let missPages: Novel = {
    title: "CSS",
}

let extraProperty: Novel = {
    pages: 300,
}
```

Errors in code

```
Property 'pages' is missing in type '{ title: str
Property 'title' is missing in type '{ pages: numl
```

extendsを使ってインタフェースの拡張ができる。

この画像をみると、NovelはWritingを拡張するのでWritingのtitleがなければならない。

```typescript
interface Writing {
    title: string;
}

interface Novel extends Writin
    pages: number;
    title: string;

}

let myNovel: Novel = {
    pages: 195,
    title: "Ant",
};

let missPages: Novel = {
    title: "CSS",
    pages: 123
}

let extraProperty: Novel = {
    pages: 300,
    title: "abc"
}
```

クラス

```typescript
class Abc {
    aa(name: string) {
        console.log("hi");
    }
}

new Abc().aa("hi");

new Abc().aa(123);
```

Errors in code

Argument of type 'number' is not assignable to parameter of type 'string'.

'name' is declared but its value is never read.

Typescriptのクラスは基本的にJavascriptと大きな違いはない。

このようにタイプを付けることができる。

```typescript
interface Learner {
  name: string;
  study(hours: number): void;
}

class Student implements Learner {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  study(hours: number): void {
    for (let i = 0; i < hours; i++) {
      console.log("....studying....");
    }
  }
}


const student = new Student("Tom");
student.study(3);
```

```
[LOG]: "....studying...."
[LOG]: "....studying...."
[LOG]: "....studying...."
```

このようにimplementsを使ってインタフェースを作ることもTypescriptにはできる。

アクセス修飾子

```typescript
class Base {
  isPublicImplicit = 0;
  public isPublicExplicit = 1;
  protected isProtected = 2;
  private isPrivate = 3;
  #truePrivate = 4;

  showInside() {
    console.log(this.isPublicImplicit);
    console.log(this.isPublicExplicit);
    console.log(this.isProtected);
    console.log(this.isPrivate);
    console.log(this.#truePrivate);
  }
}

class Subclass extends Base {
  examples() {
    this.isPublicImplicit;
    this.isPublicExplicit;
    this.isProtected;

    // this.isPrivate; // error
    // this.#truePrivate; // error
  }
}
```

```typescript
const obj = new Subclass();

// 外部
obj.isPublicImplicit;
obj.isPublicExplicit;

// obj.isProtected; // error
// obj.isPrivate; // error
// obj.#truePrivate; // error
```

アクセス修飾子をみると、ほかの言語のようにSubClassがprivateにはアクセスできない。